RESUME DBA ORACLE

Introduction : Présentation du système Oracle

On distingue deux composantes dans un système d'information : les données et les traitements.
Les données sont représentées par une base de données et les traitements par un ensemble d'applications informatiques et par des procédures manuelles. Un système de gestion de base de données (SGBD) a pour vocation de donner la possibilité de créer une base de données et de l'entretenir d'une part, et de développer des applications pour l'exploiter d'autre part. C'est un outil permettant d'insérer, de modifier et de rechercher efficacement des données spécifiques dans une grande masse d'informations partagées par les usagers.

Une base de données Oracle comprend deux types d'informations: le dictionnaire de données et les données. Le dictionnaire de données est un ensemble de tables qui décrivent la base ; les données sont de deux types : les données des utilisateurs (essentiellement les tables et les index) et les données nécessaires au fonctionnement d'Oracle (exemple les segments de rollback).
Le noyau d'Oracle réalise les fonctionnalités requises chez un SGBD, permettant ainsi d'assurer le fonctionnement normal d'une base de données (stockage des données et gestion de l'espace disque, gestion du dictionnaire de données, recherche et modification des données, sécurité des données ainsi que leur intégrité et leur confidentialités et enfin gestion de la concurrence d'accès).

I. Architecture d'Oracle

Une base de données Oracle est un ensemble de fichiers sur disque ayant chacun une structure et une vocation particulière.
Sous le terme d'architecture, il faut distinguer deux parties :

- *l'architecture physique : elle est constituée d'un ensemble de fichiers qui contiennent l'ensemble des données de la base : données de gestions de la base et données des utilisateurs. L'architecture physique d'une base Oracle comprend les fichiers de données, les fichiers de reprise et les fichiers de contrôle.*
- *L'architecture logique : elle comprend les tablespaces, les segments, les extensions, les blocs et les objets de schéma. Ces derniers comprennt les tables, les vues, les index, les procédures, les clusters, les fonctions, les triggers etc…*

Rappel :

- ***BLOC** : c'est le niveau logique le plus bas, il est composé d'un certain nombre d'octets.*
- ***EXTENSION** : c'est le niveau intermédiaire, il est constitué d'un certain nombre de blocs consécutifs alloués simultanément à un objet de schéma.*
- ***SEGMENT** : c'est le niveau supérieur, il est constitué d'un ensemble d'extensions allouées au même objet schéma.*

- **TABLESPACE** : regroupe un ensemble de segments pour faciliter leur administration

Il ne faut pas confondre une base de données Oracle et une *instance Oracle* qui est un ensemble de 5 principaux process (dont un optionnel) et une zone de la mémoire centrale appelée SGA (voir plus loin). Une instance travaille sur une et une seule base de données.

I.1 Fichiers de la base

Les fichiers de la base sont de trois types : les fichiers base (*database files*), les fichiers de journalisation (*redo log files*) et les fichiers de contrôle (*control files*)

I.1.1 Fichiers base

Ce sont les fichiers point DBL(.DBL).

Les fichiers base contiennent essentiellement deux types d'informations : les données des utilisateurs et du dictionnaire de données (DD) et les données de travail. Les premières constitent le cœur de la base de données et consistent en des tables et des index, les dernières elles sont nécessaires pour qu'Oracle puisse fonctionner.

Les fichiers base sont regroupés en unités logiques appelées **tablespace**.

Une table d'utilisateur ou du DD occupe un certain nombre de blocs oracle. Les blocs pris par une table forment ce qu'on appelle un *segment de données* ; un index occupe un *segment d'index.*

Les segments de travail représentent une partie des blocs des fichiers base ; il y a 4 types de segments de travail : les rollback segments, les segments temporaires, les segments différés et les segments du démarrage.

Les *rollback segments* sont utilés pour la réalisation du concept de transaction : les blocs de données ou d'index qui sont en cours de modification sont copiés avec leurs anciennes valeurs (l'image avant) dans un rollback segment. Si la transaction de modification se termine normalement, ces blocs d'image avant ne sont pas utiles. Par contre si la transaction est annulée ou anormalement arrêtée, ces blocs seront utilisés pour restituer l'état de la base de données avant le début de la transaction.

Les *segments temporaires* (temporary segments) sont utilisés par Oracle pour les requêtes SQL qui ne peuvent pas être réalisées en mémoire ; ces segments sont pris soit dans le tablespace système soit dans les tablespaces des utilisateurs.

Les *rollbacks segments différés* (Differed Rollback segments) sont utilisées par Oracle pour ranger des blocs « image avant » d'un tablespace lorsque celui-ci est mis hors service (Offline), et par la même occasion, l'espace qu'ils occupaient est libéré.

Le *segment de Bootstrap* (cache segment) est un segment de moins de 50 blocs qui est créé par Oracle pour ses propres besoins, lors de la création de la base.

I.1.2 Fichiers Redo log

Ce sont des fichiers de journanlisation des modifications successives, ils sont au moins au nombre de 2. Ils contiennent tous les changements d'état de la base de données et sont utiles pour la restauration d'une base suite à un incident de l'instance ou du disque. Cette restauration consiste en une ré-application du contenu des fichiers redo log sur la base.

I.1.3 Fichiers de contrôle

Ce sont les fichiers point CTL(.CTL), c'est un seule fichier Ce sont de petits fichiers qui contiennent essentiellement : les noms des fichiers base et redo log, le nom de la base, la date de création de la base et des informations sur l'état de cohérence de la base (les checkpoints). Une base doit avoir au moins un fichier de contrôle ; les autres ne sont qu'une copie conforme du premier et existent dans un but de sécurité.

I.2 Mémoire centrale

Lorsqu'une base de données Oracle est en activité, il y a quatre types de zone mémoire utilisées :

I.2.1 Zone des exécutables

Cette zone mémoire contient l'exécutable du noyau et des outils Oracle en cours d'utilisation, et des applications des utilisateurs. La taille de cette zone mémoire dépend du système d'exploitation et de la version d'Oracle. Ces exécutables peuvent être partagés par plusieurs instances à la fois sur une même machine.

I.2.2 La SGA (System Global Area)

C'est la zone mémoire réservée à l'instance; elle est allouée au démarrage et contient les informations d'une et une seule instance Oracle. Sa taille ne change pas durant l'exploitation, et dépend des valeurs de certains paramètres du fichier de démarrage INIT.ORA. Cette zone est libérée à l'arrêt de l'instance. Plus la taille du SGA est grande, plus Oracle va travailler en mémoire et moins il va accéder au disque. Les types d'informations contenues dans la SGA sont : des blocs de données et d'index des utilisateurs, des blocs du dictionnaire de données, des enregistrements *redo log* etc.
La SGA est constituée de quatre principales parties :

Pour ontenir ces composants, il suffit de lancer la commande *show sga* sous le compte system ou la requête *select * from v$sga*.

I.2.3 PGA (Programm Global Area)

Une zone est affectée à chaque process connecté à la base, le process peut être un process Oracle ou utilisateur. Cette zone dont la taille dépend du système d'exploitation contient des informations sur l'état du process et sur son activité.

I.2.4 Zones des ordres SQL

Chaque ordre SQL est stocké sous une forme particulière dans une zone mémoire appelée curseur, elle contient essentiellement le texte de l'ordre et le schéma d'accès. Il faut noter que cette zone est en dehors de la PGA de l'utilisateur et de la SGA.

I.3 Principaux processus Oracle

Les lectures de la base vers la SGA sont assurées par les processus d'arrière plan ou utilisateurs.

I.3.1 Le process DBWn

Il est responsable de l'écriture des blocs de la SGA vers la base, et plus précisément vers les fichiers de la base. process est asynchrone, car il retarde au plus l'écriture sur disque. Le process DBWn Ecrit le contenu du cache de données dans les fichiers de données :

Lorsque la place manque en SGA De façon périodique, en mettant à jour le point de restauration dans les fichiers de log

I.3.2 Le process LGWR

Il travaille conjointement avec le DBWR. Il écrit dans le fichier *redo log* pour tracer toute activité de mise à jour de la base de données. Ecrit le contenu du cache de reprise dans les fichiers de reprise :
Le process LGWr en cas de commit d'un utilisateur, Toutes les trois secondes, Quand le cache de reprise est plein au tiers, Avant qu'un processus DBWn décharge des données modifiées

I.3.3 Le process SMON

Il a pour tâche principale la restauration de la cohérence d'une base à son démarrage. Au lancement de la base, le rôle de SMON consiste à vérifier si l'instance s'était arrêtée normalement la dernière fois; auquel cas il n'a rien à faire. Sinon il restaure la base en récupérant dans le fichier *redo log* les modifications qui ont été terminées (par un *Commit* ou *Rollback*) et que Oracle n'avait pas encore écrites dans la base avant l'arrêt anormal. Ce process intervient également pour faire le ménage suite à la désactivation et à la réactivation d'une partie de la base de données (tablespace *offline* puis *online*).

I.3.4 Le process PMON

Il intervient lorsqu'un process utilisateur a un problème ou lorsqu'il est anormalement terminé. Cette intervention consiste à libérer les ressources occupées par le process utilisateur et à supprimer ce dernier de la liste des process utilisateurs de l'instance.

I.3.5 Le process ARCH

Il est optionnel et a pour rôle d'archiver les versions successives des fichiers *redo log*. Ces archives pourront être utilisées pour restaurer la base de données en cas de perte de fichiers base.

I.3.6 D'autres processus d'arrière plan

D'autres processus background pourraient être actifs dépendant des options d'installations prises :

- **CKPT** (the checkpoint process) : il est responsable de la MAJ des entêtes des fichiers base lors d'un checkpoint,
- **Le process DBWn** Il est responsable de l'écriture des blocs de la SGA vers la base, et plus précisément vers les fichiers de la base. process est asynchrone, car il retarde au plus l'écriture sur disque.
- **Le process LGWr** Il travaille conjointement avec le DBWn. Il écrit dans le fichier redo log pour tracer toute activité de mise à jour de la base de données. Ecrit le contenu du cache de reprise dans les fichiers de reprise :
- **Le process SMON** Il a pour tache principale la restauration de la cohérence d'une base à son demarrage Au lancement de la base, le rôle de SMON consiste à verifier si l'instance s'était arrêtée normalement la dernière fois ; auquel cas il n'a rien à faire Sinon il restaure la base en récupérant dans le fichier redo log les modifications qui ont été terminées (par un Commit ou Rollback) et que Oracle n'avait pas encore écrites dans la base avant l'arrêt anormal.

- **Le processus ARCH** Il est optionnel et a pour rôle d'archiver les versions successives des fichiers redo log. Ces archives pourront être utilisées pour restaurer la base de données en cas de perte de fichiers base.

II.1 Les outils de mesures : les vues

L'idéal serait que toutes les requêtes puissent s'exécuter en mémoire sans accès au disque. Or la taille de la mémoire est limitée, et les accès disque sont nécessaires. Le tuning de la mémoire consiste donc à réduire au mieux les accès disques.
Si un accès disque est nécessaire, il doit se faire au moindre coût : pas d'attente et pas de « navigation » sur le disque .
Les statistiques sur l'activité de la base sont stockées dans des tables d'information virtuelles, ; elles sont alimentées par le noyau d'Oracle et commencent par **X$.**
On peut accéder à des vues contenant les mêmes informations que ces tables ; ces vues sont décomposées en deux groupes : les vues contenant des informations en temps réel sur les utilisateurs connectés, les verrous posées, etc… :

- **v$process** : process connectés
- **v$rollname** : nom des *rollback segments* en utilisation
- **v$session** : les sessions d'utilisateurs
- **v$lock** : les verrous posés

Et les vues contenant des statistiques cumulées depuis le début du démarrage de l'instance :

- **v$rowcache** : statistiques sur le dictionnaire de données cache,
- **v$rollstat** : statistiques sur l'activité des *rollback segments*,
- **v$filestat** : statistiques sur E/S des fichiers base
- **v$waitstat** : statistiques sur la contention mémoire (les attentes)
- **v$sysstat** : statistiques cumulées pour toutes les sessions
- **v$sesstat** : statistiques sur chaque session utilisateur

II.2 Tuning de la mémoire cache

II.2.1 Optimisation du dictionnaire cache

Le dictionnaire cache est une partie du dictionnaire de données qui est en mémoire et dont la taille est déterminée par les paramètres de INIT.ORA commençant par **DC**_ Lors d'un ordre SQL, si une définition n'est pas en mémoire (*cache miss*), Oracle génère des ordres SQL qui vont chercher la définition manquante dans la base de données (*recursives calls*).

Pour examiner la valeur cumulée des *recursive calls*, visualisez **v$rowcache.**

Select parameter, gets, getmisses, count, usage from v$rowcache ;

II.2.2 Optimisation des *buffers* de la SGA

On rappelle que la SGA est une zone en mémoire qui est allouée à une instance Oracle à son démarrage. La SGA contient deux types de *buffers* : les *buffers* de données et de *rollback*, et les *buffers* de *redo log.*

- ***Buffers* de données et de Rollback**

Le paramètre **DB_BLOCK_BUFFERS** détermine la taille de la zone de ces buffers. Plus cette zone est grande, plus petite est la probabilité d'accéder au disque suite à un ordre SQL.

- ***Buffers* de Redo log**

La zone de ces buffers est une zone tampon qui contient des enregistrements qui seront écrits dans le fichier redo log courant. Pour diminuer les contentions sur cette zone, on peut augmenter cette zone en augmentant le paramètre **LOG_BUFFER** de INIT.ORA.

- **Optimisation de la mémoire pour les opérations de tri**

Oracle trie les données pour réaliser les ordres SQL qui utilisent :

- les clauses ORDER BY, GROUP BY
- la clause DISTINCT, la jointure
- la création d'index (sauf lorsque la colonne indexée est déjà triée et qu'on utilise NOSORT), et
- les opérateurs UNION, INTERSECT et MINUS.

Oracle réalise l'opération de tri dans une zone mémoire allouée au process qui a demandé le tri lorsque le volume de données à trier est faible. Avec un grand volume de données, cette zone peut s 'avérer trop petite, auquel cas Oracle utilise le segment temporaire sur disque pour stocker les résultats intermédiaires de tri, ce qui génère un surplus d'E/S.
Pour augmenter la taille de cette zone en mémoire, l'administrateur peut modifier le paramètre **SORT_AREA_SIZE**.
Cette modification peut être envisagée lorsque :

- on utilise fréquemment le tri dans les applications ;
- on est certain qu'il y a un surplus d'E/S (on peut le vérifier avec la vue v$sysstat) - on est certain que la zone agrandie tienne en mémoire.

II.3 Tuning des E/S disque

Les actions de tuning des E/S comprennent :

- répartir au maximum les données et les index sur tous les disques disponibles ; l'idéal serait que chaque tablespace soit sur un disque à part afin d'éviter la contention disque.
- Séparer les données d'une table de ses index au niveau des disques : ceci réduit aussi la contention et « parallélise » les accès disque pour une requête accédant aux tables et aux index ;
- Compacter les blocs de données et d'index en réduisant les blocs chaînés : choisir pour cela les valeurs adaptées pour les paramètres PCTFREE et PCTUSED lors de la création d'objets de la base ;

- Compacter l'étendue des objets en réduisant leur fragmentation sur plusieurs extensions : choisir pour cela les valeurs adaptées pour les paramètres INITIAL, NEXT et PCTINCREASE

Annexe1

Paramètres de stockage

INITIAL : spécifie la taille en octets de la première extension de l'objet, cette extension sera réservée automatiquement pour l'objet à sa création. Assurez vous que l'extension initiale peut contenir tout l'objet pour éviter sa fragmentation sur plusieurs extensions.
NEXT : spécifie la taille en octets de la deuxième extension d'un objet (en dehors de la première). Ce paramètre n'est pris en compte que lorsque PCTINCREASE est égal à zéro.
MINEXTENTS : spécifie le nombre minimal d'extensions qui sont allouées à l'objet à sa création (y compris l'extension initiale). Il ne peut pas être modifié pour un objet déjà créé. (Valeur par défaut
= 1)

MAXEXTENTS : spécifie le nombre maximal d'extensions qui seront allouées à un objet à sa création (y compris l'extension initiale). (Valeur par défaut = 255)
PCTINCREASE : est un pourcentage (P) qui permet de calculer la taille (TN) en octets d'une nouvelle extension par rapport à la taille de la dernière extension créée (TD).
Formule de calcul : TN=TD + TP x TD

(Valeur par défaut = 50%)

PCTFREE : spécifie le pourcentage à réserver en tant que zone de débordement dans l'espace utile du bloc. (Valeur par défaut 10%)
PCTUSED : spécifie le pourcentage de la place utilisée par les lignes du bloc au dessous duquel un bloc peut de nouveau être utilisé pour insérer de nouvelles lignes.
(Valeur par défaut = 40%) .

INITRANS, MAXTRANS : nombre minimum et maximum de transactions en concurrence sur l'objet.

TP1 sous Oracle

I Constituants de la base et paramètres de la SGA

A. Fichiers de la base/rollback segment

- Afficher les fichiers base (v$datafile), les fichiers redo log (v$logfile) et les fichiers de contrôle (v$controlfile)
- afficher les noms des *rollback segment* et leur statut (dba_rollback_segs)
- afficher la liste des tablespaces et des fichiers associés (dba_data_files, dba_tablespaces)

I. Rq : Décrire les tables concernées pour choisir les colonnes à afficher

B. Afficher les valeurs des paramètres de la SGA

A partir de la vue **v$sga**, ou à partir de **show**, visualiser les paramètres de la SGA.

II Redirection des résultats dans des fichiers

A. Lister les rollback segments et leur statut et stocker le résultat dans un fichier *rollback.txt*

B. On veut nettoyer une base en détruisant toutes les tables exceptées celles appartenant à

SYS et à **SYSTEM**. Ecrire une requête SQL générant ce script (le script s'appellera *drobtab.sql*)

C. Afficher toutes les variables d'environnement, le résultat est stocké dans le fichier *var.txt*.

III. Plan d'exécution des requêtes SQL

a) Créer une table *audit_sql* permettant d'enregistrer les plans d'exécution fournis par EXPLAIN PLAN

b) Créer une requête permettant d'exploiter le contenu de la table *audit_sql*

c) Utiliser EXPLAIN PLAN pour obtenir le plan d'exécution de la requête donnée en b)

IV Example d'un fichier INIT.ORA

(partie sautée)

Correction TP2

I Constituants de la base et paramètres de la SGA

A. Fichiers de la base/rollback segment

Fichiers base (v$datafile)

- *SQL> select name, status, enabled from v$datafile;*

fichier redo log (v$logfile)

- *SQL> select * from v$logfile;*

fichier de contrôle

- *select member from v$controlfile;*

afficher les noms des rollback segment et leur status (dba_rollback_segs)

- *SVRMGR> select segment_name, owner, status from dba_rollback_segs;*

```
SEGMENT_NAME                   OWNER      STATUS
------------------------------ ---------  ----------------
SYSTEM                         SYS            ONLINE
R01                            SYS            ONLINE
R02                            SYS            ONLINE
R03                            SYS            ONLINE        R04
SYS             ONLINE
5 rows selected.
```

afficher la liste des tablespaces et des fichiers associés (dba_data_files, dba_tablespaces)

- *SQL> select t.tablespace_name tablespace,*

  2> f.file_name fichier

  3> from dba_tablespaces t,

  4> dba_data_files f

  5> where f.tablespace_name=t.tablespace_name;

B Paramètres de la SGA

A partir de la vue **v$sga**, visualiser les paramètres de la base

- *SVRMGR> show sga*

  Ou

- *SQL> select * from v$sga;*

II Redirection des résultats dans des fichiers

Rollback segments dans rollback.txt

- *SQL> spool rollback.txt*
- *SVRMGR> select segment_name, owner, status from dba_rollback_segs;*
- *SQL> spool off*

B. Nettoyage de la base

- *SQL> set heading off*
- *SQL> spool droptab.sql*
- *SQL> select 'drop table ' || owner || '.' || table_name || '/' from dba_tables where owner not in ('SYS','SYSTEM');*
- *SQL> spool off*

C. Variables dans var.txt

- *SQL> spool var.txt*
- *SQL> Show all*
- *SQL> spool off*

III Plan d'exécution des requêtes SQL

a) Création de la table contenant le plan d'exécution

(tabaudit.sql)

```
CREATE TABLE audit_sql(
statement_id    char(30),
timestamp       date,
remarks         char(80),
operation       char(30),
options         char(30),
object_node     char(30),
object_owner    char(30),
object_name     char(30),
object_instance number,
object_type     char(30),
search_columns  number,
id              number,
parent_id       number,
position        number,
other           long
);
```

b) Exemple de requête dont le plan est stocké dans la table audit_sql (explreq.sql)

```
EXPLAIN PLAN
SET STATEMENT_ID='1'
INTO audit_sql
FOR
SELECT TABLESPACE_NAME, USERNAME, BYTES, MAX_BYTES
FROM dba_ts_quotas
WHERE USERNAME in ('MASTER1','MAITRISE2', 'LICPRO1', 'MOUSSA')
ORDER BY USERNAME;
```

c) affichage de certaines colonnes décrivant le plan (myaudit.sql)

```
SET  ECHO  OFF
SET   VER   OFF
spool resultat.txt
prompt EXAMEN DES PLANS D'EXECUTION DES REQUETES
prompt TP Maîtrise Infomartique prompt
******************
prompt            prompt Entrer la reference de la
requete analysee: accept val prompt "->" col op
heading operation format a23 col objn heading
"table|/index" format a25
```

```
col obi heading "nu.|objet" format 99990
col id format 99 col pid heading p_id
format 9990 col p heading ordre format
99990

SELECT operation ||' '|| options op
,object_name ||' '|| object_type objn
,object_instance obi
,id
,parent_id
,position p
FROM audit_sql
WHERE statement_id='&val'
ORDER BY id;
```

V. spool off

Remarque : - On peut aussi regrouper le b) et le c) dans un même script

- Consulter le fichier resultat.txt

DEUXIEME PARTIE

VI. Création/démarrage d'une base de donnée

La préparation d'un fichier d'initialisation (en général initNOMBASE.ORA) est nécessaire pour créer une base sous Oracle. Ce fichier contient tous les paramètres ainsi que leurs valeurs (nom de la base, nom de l'instance, la liste des fichiers de contrôle, le nom du domaine etc…)

I.1 Création d'une base de données Pour créer une base, il faut deux étapes :

- installer le logiciel Oracle
- Créer la base avec la commande create database.

Syntaxe de la commande create databse:

create database NomBaseDeDonnees
[controlfile reuse] (lors d'une suppreesion suivie d'une création d'une BD)
[logfile fichier1 [,fichier2]…] (définit les fichiers de reprise « rdo log file)
[datafiles fichierA [,fichierB]…] (définit les fichiers bases)
[maxlogfiles valeur] (le nombre maximum de fichiers de reprise)
[maxdatafiles valeur] (le nombre max de fichiers de données)
[maxinstances valeur] (le nombre max d'instances pouvant accéder simultanément à la base)

[archivelog | noarchivelog] (positionne le fonctionnement en mode archivage| non archivage)

I.2. Démarrage de la base

La mise en œuvre d'une base de données s'effectue en 3 étapes par utilisation des commande STARTUP et ALTER DATABASE sous DBA.

1. démarrer l'instance (initialisation de l'environnement de la BD en allouant les ressources) : la base est dans l'état *démarré non montée* (NOMOUNT).

Exemple : SQLDBA> STARTUP NOMOUNT

2. le démarrage de la base qui consiste à associer une base à l'instance créée à l'étape 1. La base est accessible en mode INTERNAL par les utilisateurs qui ont les privilèges d'administration. Et qui peuvent effectuer des opérations de maintenance telles que renommer les fichiers de la base ou gérer les fichiers de reprise.

Exemple : SQLDBA> CONNECT / as sysdba SQLDBA> ALTER DATABASE *nom_base* MOUNT

3. Ouverture de la base pour rendre ses données accessibles aux utilisateurs. La base est à l'état *ouvert* OPEN.

Exemple : SQLDBA> ALTER DATABASE *nom_base* OPEN

I.3 Arrêt de la base

La fermeture s'effectue en trois étapes :

1. Déconnexion des utilisateurs de la base
2. Détachement de la base de l'instance
3. Arrêt de l'instance

Pour arrêter une base utiliser l'ordre : **SHUTDOWN**
[*option*]
Où ***option*** peut prendre les valeurs :

- NORMAL : aucune nouvelle connexion permise, terminaison normale des transactions et arrêt lorsque le dernier utilisateur se déconnecte
- IMMEDIATE : arrêt immédiat sans attente de la fin des transactions en cours, qui sont alors avortées.
- ABORT : arrêt immédiat sans détachement de la base ni déconnexion préalable des utilisateurs. C'est un mode de fermeture rapide mais nécessitant une reprise lors du redémarrage de la base
-

I.4 Ajout de fichiers, de tablespaces dans la base

I.4.1 Ajout d'un fichier de contrôle

Pour ajouter un fichier de contrôle (par exemple *CONTROLE04.CTL*), il faut suivre la procédure suivante :

- arrêter l'instance
- copier avec l'OS un fichier de contrôle existant et le renommant sous le nom de

CONTROLE04.CTL

- ajouter dans le paramètre control_files du fihier d'initialisation ce nouveau fichier de contrôle puis enregistrer.
- Démarrer la base avec **startup**.

I.4.2 Ajout/Suppression d'un fichier log et d'un nouveau groupe

Pour ajouter un fichier de log (par exemple log4.ORA), on suit les étapes suivantes :

- arrêter l'instance
- démarrer la base en mode mount • lancer la commande

alter database add logfile member 'Chemin_complet_du_fichier\log4.ora' to groupe Numero_groupe

- Vérifier l'état du fichier ajouté avec *select * from v$logfile*
- Rendre la base ouverte
- Lancer autant de fois que c'est nécessaire l'ordre *alter system switch logfile*
- Vérifier encore l'état du fichier log ajouté.
- Pour ajouter un nouveau groupe (4) on lance :

ALTER DATABASE ADD LOGFILE GROUP 4 'Chemin_complet_du_fichier\ReDO03.LOG' size 10M;

- Pour supprimer le groupe 4 on lance :

ALTER DATABASE DROP LOGFILE GROUP 4;

- Pour supprimer un fichier d'un groupe on lance :

ALTER DATABASE DROP LOGFILE MEMBER 'g:\oracle\oradata\orafrance\REDO02.LOG';

I.4.3 Ajout d'un fichier base dans un tablespace

Pour ajouter un fichier base (par exemple user05.ora) il suffit de lancer l'ordre *alter tablespace Nom_tablespace add datafile 'Chemin_complet\user05.ora' size* ***taille_init K|M*** Pour modifier la taille d'un fichier de donnée on lance :
ALTER DATABASE DATAFILE 'g:\oracle\oradata\orafrance\ORA_DATA02.dbf' ***RESIZE*** *50M;*

I.4.4 Ajout d'un tablespace

Pour ajouter un tablespace (par exemple DEV) il suffit de lancer la commande :
create tablespace DEV datafile 'Chemin_complet\Nom_fichier' size taille_init default storage (initial valeur K|M next valeur K|M pctincrease valeur minextents valeur maxextents valeur|unlimited)

I.4.5 Déplacement d'un tablespace

Pour déplacer le tablespace dev:

1°) Mettre le tablespace OFFLINE :

ALTER TABLESPACE dev OFFLINE;

2°) Copier le fichier dans le nouveau répertoire **3°)** Renommer le fichier :
ALTER DATABASE RENAME FILE 'g:\oracle\oradata\fstm\dev01.dbf' TO 'g:\oracle\oradata\fstm\sene\dev01.dbf; **4°)** Mettre le tablespace ONLINE :
ALTER TABLESPACE dev ONLINE;

5°) Supprimer le fichier g:\oracle\oradata\fstm\dev01.dbf.

Pour mettre le tablespace toolsen lecture seule :

ALTER TABLESPACE tools READ ONLY; Pour le mettre en lecture/écriture :
ALTER TABLESPACE tools READ WRITE; Pour supprimer un tablespace :

DROP TABLESPACE tools INCLUDING CONTENTS;

NB: Cette commande ne supprime pas le datafile, il convient donc de supprimer le fichier haddock si nécessaire.

I.4.6 L'UNDO management

La gestion des annulations automatiques (AUTOMATIC UNDO MANAGEMENT en opposition au MANUAL ie. gestion des RBS), se fait en plusieurs étapes.

La première consiste à créer un tablespace d'annulation. Par exemple, pour un tablespace UNDOTBS01 géré localement :

CREATE UNDO TABLESPACE undotbs01 DATAFILE 'Chemin_Absolu\undotbs01.dbf' size 500M REUSE AUTOEXTEND OFF /

Puis, il nous faut modifier le paramètrage de notre base. Dans un premier temps il faut stipuler le fait que l'on va travailler en automatique avec notre tablespace UNDOTBS01 (si on travaille avec un SPFILE, voila ce que cela donnera) :

ALTER SYSTEM SET UNDO_MANAGEMENT=auto SCOPE=SPFILE;

ALTER SYSTEM SET UNDO_TABLESPACE=undotbs01 SCOPE=SPFILE;

Il nous faut ensuite déterminer le temps de conservation de nos données. En gros, on va dire au système combien de secondes on désire conserver nos données pour annulation.

Ce paramètre se nomme UNDO_RETENTION.
Si par exemple, on règle ce paramètre à 10 minutes (soit 600 secondes), alors on considère que toutes nos donéées d'annulations pourront être conservées pendant 10minutes ... au delà, une erreur caractéristique se produira ... l'ORA-01555 (voir plus bas).

ALTER SYSTEM SET UNDO_RETENTION=600 SCOPE=SPFILE; Enfin, on redémarre l'instance pour la prise en compte.

II Gestion des utilisateurs

II.1 Création

Pour créer un utilisateur, on utilise la commande suivante suivante :

CREATE USER nom_utilisateur

IDENTIFIED BY mot_de_passe

[DEFAULT TABLESPACE nom_tablespace]

[TEMPORARY TABLESPACE nom_tablespace]

[QUOTA [nombre | UNLIMITED] ON nom_tablespace] (limitation ou non d'espace)

[PASSWORD EXPIRE] (demande le changement du mot de passe)

[ACCOUNT {LOCK | UNLOCK}] (Compte bloqué | non bloqué)

II.2 Modification

Pour modifier un utilisateur, utilisez

ALTER USER nom_utilisateur

IDENTIFIED BY mot_de_passe

[DEFAULT TABLESPACE nom_tablespace]

[TEMPORARY TABLESPACE nom_tablespace]

[PASSWORD EXPIRE] (demande le changement du mot de passe)

[ACCOUNT {LOCK | UNLOCK}] (Compte bloqué | non bloqué)

II.3 Suppression

La suppression d'un utilisateur s'effectue par l'ordre :

DROP USER schéma [CASCADE]

Sans l'option CASCADE, l'utilisateur n'est pas supprimé s'il possède des objets.

Atelier 1

1. Créer un utilisateur **toto** (avec comme mot de passe **toto**) sans le rattacher à un tablespace et vérifier par la table **dba_users** si l'utilisateur toto existe bien.
2. A quel tablespace le système associe l'utilisateur toto (examiner la vue dba_users)
3. Lister l'ensemble des tablespaces et choisir **users** comme tablespace par défaut et **temp** comme tablespace temporaire.

4. L'utilisateur toto peut se connecter, créer ses tables ? Si oui essayer de vous connecter sous toto et créer une table T (nom char(20)) sinon dites pourquoi.
5. Le nouveau mot de passe de toto est **maitrise**, faites la modification.

III Contrôle des accès

Ce contrôle comporte plusieurs niveaux :

- Gestion des accès à la base de données
- Gestion des accès aux données de la base
- Limitation des ressources accessibles aux utilisateurs
- Attribution d'accès par défaut

L'accès à la base s'effectue par l'intermédiaire de la notion d'utilisateur (compte oracle) ; chaque USER est défini par son nom, son mot de passe, un ensemble de privilèges et profil. A la création de la base, deux utilisateurs sont créés : SYSTEM (qui permet de créer les autres USER) et SYS qui est propriétaire de la majorité des tables du dictionnaire ainsi que des vues crées sur ces tables.
Un privilège est le droit attribué à un utilisateur d'exécuter un ensemble particulier d'ordres SQL ou d'accéder à certains objets de la base; il peut être au niveau système (autorisation d'effectuer une action particulière) ou au niveau objet (autorisation donnée par le créateur de l'objet à un autre utilisateur). Il faut noter que le privilège peut être attribué soit directement à un utilisateur soit à un rôle. Un rôle est un groupement de privilèges qui peut être attribué
Cinq rôles sont définis dans Oracle8 :

- **CONNECT** : l'utilisateur peut se connecter à la base de données et manipuler les objets de la base pour lesquels ils ont préalablement eu un droit de manipulation (lecture, modification, insertion, suppression)
- **RESOURCE** : l'utilisateur peut créer ses propres tables, index, procédures, fonctions et déclencheurs.
- **DBA** : l'utilisateur possède tous les privilèges système nécessaires à l'administrateur de la base de données
- **EXP_FULL_DATABASE** : l'utilisateur peut exporter la base de données
- **IMPORT_FULL_DATABASE**: l'utilisateur peut importer la base de données

III.1 Gestion des privilèges systèmes

A Création

GRANT {Privilège niveau système | rôle}[,Privilège niveau système |rôle}]…

TO {nom utilisateur | rôle | PUBLIC}[,nom utilisateur | rôle | PUBLIC]…

[WITH ADMIN OPTION] (autorise la transmission du privilège) PUBLIC
désigne tous les utilisateurs.

B. Suppression

REVOKE {Privilège niveau système | rôle}[,Privilège niveau système |rôle}]…

FROM {nom utilisateur | rôle | PUBLIC}[,nom utilisateur | rôle | PUBLIC]…

C. Quelques privilèges systèmes

- ALTER ANY TABLE
- ALTER DATABASE
- ALTER SESSION
- DROP ANY TABLE
- SELECT ANY TABLE -DELETE ANY TABLE
- LOCK ANY TABLE
- CREATE ANY SYNONYM
- ALTER TABLESPACE

III.2 Gestion des privilèges objets

A Création

GRANT Privilège niveau objet[,Privilège niveau objet]…

ON [schéma.]objet

TO {nom utilisateur | rôle | PUBLIC}[,{nom utilisateur | rôle | PUBLIC]…. [WITH GRANT OPTION] (autorise la transmission du privilège) où objet peut être une table, une vue, une séquence, une procédure, une fonction, un package, une librairie, un type

B. Suppression

REVOKE Privilège niveau objet[,Privilège niveau objet] | ALL PRIVILEGES}

ON objet

FROM {nom utilisateur | rôle | PUBLIC}[,nom utilisateur | rôle | PUBLIC]…

[CASCADE CONSTRAINTS] (utilisables avec references)

[FORCE] (utilisable avec EXECUTE pour les objets ayant des dépendances sur des tables)

C. Quelques privilèges objets

- SELECT [(col,...)]
- INSERT
- UPDATE [(col,...)]
- DELETE
- EXECUTE (droit d'exécuter une procédure, une fonction ou package)

- ALTER
- READ
- ALL (tous les droits)

Remarque:

La confidentialité est la vocation première d'une vue ; ainsi pour empêcher à un utilisateur de consulter des colonnes ou des lignes d'une table, le moyen le plus simple est de créer des vues contenant ces colonnes ou lignes.
Soit la table T(col1, col2, col3, col4) et l'utilisateur U auquel on veut indire l'accès aux colonnes col3 et col4 :

- CREATE VIEW VUE_T AS SELECT col1, col2

FROM T;

- GRANT SELECT ON VUE_T TO U;

III.3 Audit de contrôle d'accès

Il peut s'avérer intéressant d'observer l'activité de la base en un laps de temps par exemple pour quantifier le nombre de connexions à la base et leur type. L'administrateur pourrait être intéressé de savoir le nombre d'utilisateurs connectés durant une période donnée, les tables accédées, les terminaux à partir desquels il y a eu tentatives de connexions. Ce type d'observation v$filestat, dit AUDIT, consiste à garder dans le dictionnaire de données une trace de certaines actions sur la base ; son intérêt se reflète à trois niveaux :

- étudier la charge de travail,
- renforcer la sécurité,
- établir des statistiques

On distingue deux types d'audit :

- les audits dits systèmes effectués par le DBA,
- les audits effectués par les propriètaires d'objets

A. Audit système

Il ne peut être lancé que par le DBA :

- les connexions et déconnexions à/de la base (AUDIT CONNECT) sont contenus dans la vue **DBA_AUDIT_CONNECT**
- les ordres GRANT, REVOKE, AUDIT, NOAUDIT etc. (AUDIT DBA) sont visibles dans la vue **DBA_AUDIT_DBA**,
- les références à des objets non existants (AUDIT NOT EXISTS) ont leur résultats dans la vue **DBA_AUDIT_EXISTS**

- les ordres de création, de suppression et de modification de tables, de tablespace, de Rollback segments, etc… (AUDIT RESSOURCE) ont leurs résultats dans la vue **DBA_AUDIT_RESSOURCE**.

Ces vues contiennent essentiellement la date de l'action, le nom de l'utilisateur, le type de l'action, le résultat de l'action (succées ou échec), le nom de l'objet créé (ordre de création) etc. L'arrêt d'un audit s'effectue par la commande NOAUDIT.

Exemples :

- AUDIT CONNECT WHENEVER SUCCESFULL permet de tracer toutes les connexions et déconnexions réussies
- AUDIT NOT EXISTS permt de tracer les ordres SQL faisant référebnce à des objets non existants.
- NO AUDIT ALL arrête toutes les traces d'audit.
- AUDIT CREATE TABLE BY maitriseX: audite tous les ordres create table lancés par l'utilisateur maitriseX
- AUDIT USER ; Audit tous les ordres CREATE USER, ALTER USER ou DROP USER lancés par n'importe quel utilisateur

B. Audit des propriétaires d 'objets

Un utilisateur peut auditer toutes les actions effectuées sur ses tables. Il peut examiner les actions de lecture, de MAJ, d'indexage etc en scrutant la vue **USER_AUDIT_TRAIL**.
Pour déclencher l'audit, il doit lancer :

AUDIT option1, option2,….ON table1, table2,…

Les options peuvent être : ALTER, INSERT, SELECT, UPDATE, DELETE, LOCK, AUDIT, GRANT etc.

Exemples :

- AUDIT SELECT, INSERT ON Client BY ACCESS: trace tous les ordres SELECT et INSERT sur la table Client
- AUDIT ALL ON DEFAULT BY ACCESS : trace toutes les opérations sur toutes les tables qui seront créées
- NOAUDIT ALL ON DEFAULT : Les tables qui seront créées ne seront pas ausitées ; cet ordre ne concerne pas les tables déjà auditées.
- Pour arrêter les audits sur des tables en cours d'audit (par exemple Commande, Produit), il faut lancer NOAUDIT ALL ON Commande, Produit .

Atelier 2

1. On suppose que vous avez deux comptes oracles sur votre machine : **moussa** et **toto**. En vous connectant sous **moussa**, accordez le droit de **SELECT** à toto sur la table **Client**, le droit de **DELETE** sur la table **Commande** avec possibilité de transmission et le droit de **INSERT** sur la table **Produit**.
2. Examiner les vues **DBA_USERS** et **DBA_TAB_GRANTS**
3. Lister le profil des utilisateurs en utilisant la vue **DBA_USERS**
4. Lister les privilèges sur vos tables en utilisant la vue **DBA_TAB_GRANTS**

5.